

Proyecto Final - Base de Datos Gimnasio

1. Introducción

El presente proyecto consiste en el diseño de una base de datos para la gestión integral de un gimnasio. La solución permite administrar socios, pagos, asistencias, rutinas y ejercicios, garantizando integridad y trazabilidad de la información.

2. Objetivo

Desarrollar un esquema relacional que soporte la operatoria del gimnasio, contemplando información administrativa, control de membresías, seguimiento de rutinas y análisis de asistencia.

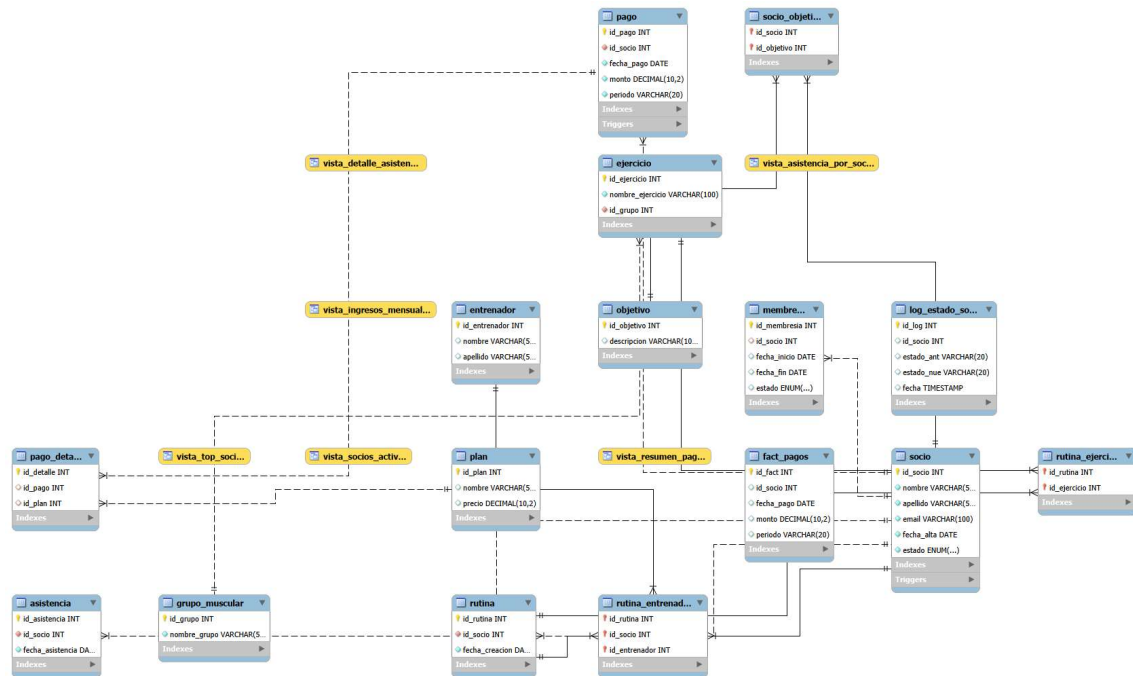
3. Situación Problemática

La gestión manual o desestructurada de datos genera inconsistencias, pérdida de información y dificultad para obtener métricas operativas. La implementación de una base de datos relacional resuelve estas brechas mediante normalización y control de relaciones entre entidades.

4. Modelo de Negocio

El gimnasio opera mediante socios activos que abonan periódicamente una membresía. Cada socio puede registrar asistencias y tener rutinas asignadas compuestas por múltiples ejercicios.

5. Diagrama Entidad-Relación



6. Script SQL

El proyecto incluye un archivo SQL con la creación de la base de datos y sus tablas, incluyendo claves primarias, foráneas e índices correspondientes.

A continuación se describen las entidades que componen el modelo de datos del sistema:

Tabla: Socio

- id_socio (INT, PK, AUTO_INCREMENT)
- nombre (VARCHAR(50))
- apellido (VARCHAR(50))
- email (VARCHAR(100), UNIQUE)
- fecha_alta (DATE)
- estado (ENUM: ACTIVO / INACTIVO)

Tabla: Pago

- id_pago (INT, PK, AUTO_INCREMENT)
- id_socio (INT, FK → Socio.id_socio)
- fecha_pago (DATE)
- monto (DECIMAL(10,2))
- periodo (VARCHAR(20))

Tabla: Asistencia

- id_asistencia (INT, PK, AUTO_INCREMENT)
- id_socio (INT, FK → Socio.id_socio)
- fecha_asistencia (DATE)

Tabla: Grupo_Muscular

- id_grupo (INT, PK, AUTO_INCREMENT)
- nombre_grupo (VARCHAR(50), UNIQUE)

Tabla: Ejercicio

- id_ejercicio (INT, PK, AUTO_INCREMENT)
- nombre_ejercicio (VARCHAR(100))
- id_grupo (INT, FK → Grupo_Muscular.id_grupo)

Tabla: Rutina

- id_rutina (INT, PK, AUTO_INCREMENT)
- id_socio (INT, FK → Socio.id_socio)
- fecha_creacion (DATE)

Tabla: Rutina_Ejercicio

- id_rutina (INT, PK, FK → Rutina.id_rutina)
- id_ejercicio (INT, PK, FK → Ejercicio.id_ejercicio)

Tabla: Membresia

- id_membresia (INT, PK, AUTO_INCREMENT)
- id_socio (INT, FK → Socio.id_socio)
- fecha_inicio (DATE)
- fecha_fin (DATE)
- estado (ENUM: ACTIVA / VENCIDA)

Tabla: Plan

- id_plan (INT, PK, AUTO_INCREMENT)
- nombre (VARCHAR(50))
- precio (DECIMAL(10,2))

Tabla: Pago_Detalle

- id_detalle (INT, PK, AUTO_INCREMENT)
- id_pago (INT, FK → Pago.id_pago)
- id_plan (INT, FK → Plan.id_plan)

Tabla: Entrenador

- id_entrenador (INT, PK, AUTO_INCREMENT)
- nombre (VARCHAR(50))
- apellido (VARCHAR(50))

Tabla: Rutina_Entrenador

- id_rutina (INT, PK, FK → Rutina.id_rutina)
- id_socio (INT, PK, FK → Socio.id_socio)
- id_entrenador (INT, FK → Entrenador.id_entrenador)

Tabla: Objetivo

- id_objetivo (INT, PK, AUTO_INCREMENT)
- descripcion (VARCHAR(100))

Tabla: Socio_Objeto

- id_socio (INT, PK, FK → Socio.id_socio)
- id_objetivo (INT, PK, FK → Objetivo.id_objetivo)

Tabla: fact_pagos (Tabla de Hechos)

- id_fact (INT, PK, AUTO_INCREMENT)
- id_socio (INT)
- fecha_pago (DATE)
- monto (DECIMAL(10,2))
- periodo (VARCHAR(20))

7. Listado de Vistas

- **vista_detalle_asistencia:**
 - **Descripción:** Presenta un reporte de asistencia que vincula la identidad de los socios con sus fechas de ingreso.
 - **Objetivo:** Facilitar al personal de recepción el control de flujo diario de clientes.
 - **Tablas:** Socio, Asistencia.
- **vista_resumen_pagos:**
 - **Descripción:** Consolida el historial financiero de cada socio, sumando todos los montos abonados.
 - **Objetivo:** Identificar rápidamente el valor total aportado por cada cliente y su frecuencia de pago.
 - **Tablas:** Socio, Pago.
- **vista_ingresos_mensuales**
 - **Descripción:** Agrupa los ingresos generados por mes.
 - **Objetivo:** Permitir el análisis financiero periódico del gimnasio.
 - **Tablas:** Pago.
- **vista_top_socios**
 - **Descripción:** Ranking de socios según el monto total abonado.
 - **Objetivo:** identificar clientes de mayor valor.
 - **Tablas:** socio, pago
- **vista_asistencia_por_socio**
 - **Descripción:** total de asistencias registradas por socio.
 - **Objetivo:** Medir el nivel de uso de las instalaciones.
 - **Tablas:** Asistencia
- **vista_socios_activos:**
 - **Descripcion:** lista de socios actualmente activos.
 - **Objetivo:** facilitar la gestión operativa.
 - **Tablas:** socio.

8. Listado de Funciones

- **fn_nombre_grupo_muscular:**
 - **Descripción:** Recibe el identificador de un ejercicio y retorna el nombre del grupo muscular asociado.
 - **Objetivo:** Simplificar la lectura de rutinas para el usuario final evitando IDs numéricos.
 - **Datos/Tablas:** Implementada sobre Ejercicio y Grupo_Muscular.
- **fn_asistencias_mes:**
 - **Descripción:** Calcula el total de asistencias de un socio específico dentro del mes calendario actual.
 - **Objetivo:** Generar métricas de fidelización y uso de las instalaciones.
 - **Datos/Tablas:** Manipula la tabla Asistencia.
 -

o Nota: En caso de no encontrar resultados, la función retorna un valor por defecto.

9. Listado de Stored Procedures

- **sp_registrar_socio_con_pago:**
 - **Descripción:** Ejecuta de forma atómica el alta de un nuevo socio y el registro de su pago inicial.
 - **Objetivo:** Asegurar la integridad de los datos, garantizando que no existan socios activos sin una membresía pagada.
 - **Tablas:** Socio, Pago.
- **sp_registrar_pago:**
 - **Descripción:** Registra un nuevo pago para un socio existente
 - **Objetivo:** Facilitar la gestión de cobranzas.
 - **Tablas:** Pago.

10. Listado de Triggers

- **tr_auditoria_estado_socio:**
 - **Descripción:** Disparador que se activa ante cualquier cambio en la columna estado de la tabla Socio.
 - **Objetivo:** Registrar un historial de auditoría cada vez que un socio pasa de ACTIVO a INACTIVO o viceversa.
 - **Tablas/Situaciones:** Se acciona sobre la tabla Socio antes de una actualización (BEFORE UPDATE).
- **Trg_fact_pagos:**
 - **Descripción:** Disparador que inserta automáticamente los pagos en la tabla de hechos
 - **Objetivo:** alimentar la capa analítica sin intervención manual
 - **Tablas:** Pago, fact_pagos

11. Capa analítica.

Se implementó una tabla de hechos denominada fact_pagos, la cual almacena los registros de pagos de manera desnormalizada para facilitar consultas analíticas.

Esta tabla es alimentada automáticamente mediante el trigger trg_fact_pagos, permitiendo separar la lógica transaccional de la analítica.

Gracias a esta estructura, se pueden generar reportes como ingresos mensuales, ranking de clientes y comportamiento de pagos sin afectar el rendimiento de las tablas principales.

12. Herramientas y Tecnologías

Se utilizó MySQL Workbench para el diseño y desarrollo de la base de datos.

Para el análisis de datos, se pueden utilizar herramientas como Microsoft Excel o Power BI, conectando la tabla fact_pagos para generar reportes de ingresos, tendencias mensuales y comportamiento de clientes. Se adjunta un ejemplo de exportación.

Paso a Paso para la Importación:

1. **Preparación del Archivo:** Asegurar que el archivo de origen (ej. socios_nuevos.csv) contenga cabeceras que coincidan con los nombres de las columnas en la tabla destino (ej. nombre, apellido, email).
2. **Asistente de Importación:** En el panel *Schemas*, hacer clic derecho sobre la tabla deseada y seleccionar **"Table Data Import Wizard"**.
3. **Selección de Origen:** Navegar y seleccionar el archivo local a importar.
4. **Configuración de Mapeo (Destino):**
 - Verificar que las columnas del CSV se vinculen correctamente con los campos de la tabla en gimnasio_db.
 - **Nota:** Si la tabla tiene un id con AUTO_INCREMENT, se recomienda dejar ese campo sin mapear para que el motor asigne los IDs automáticamente.
5. **Ejecución:** Confirmar la importación. El asistente procesará las filas y notificará si hubo errores de restricción (ej. correos duplicados por la restricción UNIQUE en la tabla Socio).
6. **Verificación:** Ejecutar un SELECT * FROM [tabla] para confirmar que los registros se visualizan correctamente en el sistema.